Adaptive HACT: Information Limits and Safe Adaptation of Verification Evidence

Da Xu and Fei Wang

Abstract—Adaptive verification evidence raises two questions: which completion statistics identify an efficient representation, and how may that representation change without weakening acceptance? Adaptive HACT (Hyperedge-Aware Certificate Trees) separates a protected semantic registry from generation-fenced certificate trees and prices packets touched by joint completion sets. We prove that exact singleton and pairwise probabilities can leave globally optimal trees unidentified for every fanout of at least three and every affine positive-slot packet price, with sharp minimax regret. A complete-epoch construction extends this limitation to fixed-eight batching within a two-layout class. Under exchangeable complete batches, a specialization of established tree optimization provides an exact global baseline. A local publication invariant preserves acceptance through migration; a conditional adoption rule includes calibration, shift and installation costs. A sparse compact compiler admits a 26,948-check registry within 320 MiB in a repeated-pattern replay. Retrospective scoring finds that the symmetric baseline beats frozen full-wave refinement on all 15 item-shuffled traces and loses on all 15 file-group traces. It costs 21.26–62.13% more on original holdouts. Without byte observers, the tree service takes 1.61–3.54 times the maintained-counter ledger's operation time. The results establish information limits and conditional adaptation guarantees; service and agent speedups remain unproven.

Index Terms—Verification evidence, software agents, completion hypergraphs, adaptive data structures, runtime assurance.

1 Introduction

Software agents navigate repositories, run checks and revise candidates. SWE-bench, SWE-agent and OpenHands expose interactions with tooling [1], [2], [3], while verification-first approaches bind actions to checkable evidence [4], [5]. Even with a fixed checker and acceptance criteria, how can an orchestrator reorganize evidence without changing what it authorizes?

An indexed ledger can report coverage and failures, replay completion batches and serve bounded diagnostic pages. A hierarchy must repay its construction, maintenance and adaptation, and a prescribed packet format must serve a justified consumer need.

Adaptive HACT (Hyperedge-Aware Certificate Trees) separates a protected semantic registry from a replaceable certificate tree. Acceptance requires current trusted records for all declared checks. Advisory layouts change only representation; a monotone generation fences publication through migration, including a return to an earlier layout.

The objective depends on completion *sets*: a packet is emitted once when its coverage intersects a set. Pairwise co-activity can miss the higher-order event that touches a subtree. We construct identical pairwise observations with disjoint globally optimal trees across a family of prices and fanouts, and quantify the sharp minimax penalty of pairwise information. The fixed-eight extension affects a local refinement decision when every check completes once; it does not establish global optimality of the embedded layouts. Table 1 distinguishes established ingredients.

Under a uniform completion permutation divided into fixed-size batches, expected cost depends only on coverage cardinality. A size-state dynamic program optimizes all shapes and permutations in $O(bn^2)$ arithmetic operations after computing the weights. This instance of established batch-update tree optimization [6] gives a strong null baseline: learning cannot improve expected cost when the future law remains exchangeable conditional on training.

Full-wave compilation may offer little improvement over a compact tree, and a hierarchical service may lose to maintained ledger counters. We test both without treating analytical separation as an execution speedup. Our four contributions are:

1) A conditional acceptance invariant covering stable identities, conflicts and generation-fenced migration.
2) A packet-refresh model, parameter-general information lower bound with sharp deterministic and randomized regret, and fixed-batch separation.
3) An exact exchangeable reference, bounded full-wave refinement and an adoption rule accounting for paired-cost bounds, calibration and installation.
4) A reproducible evaluation of historical representation results, new test executions and controlled service demand, with strong ledger controls and explicit resource failures.

We claim no priority for hyperedges or hierarchies generally; the contribution is their composition under the stated contract and packet objective.

2 Model and Measurement Contract

Let $\mathcal{R} = (g_0, \ldots, g_{n-1})$ be a nonempty registry of unique semantic check IDs. A candidate identity $e = (\mathrm{epoch}, \mathrm{snapshot}, \mathrm{checker}, \mathrm{environment}, \mathrm{registry})$ binds the source and protected acceptance inputs. The epoch increases

• *The authors are with China Mobile Research Institute, Beijing, China. Corresponding author: Da Xu. E-mail: xudayj@chinamobile.com; wangfei@chinamobile.com.*

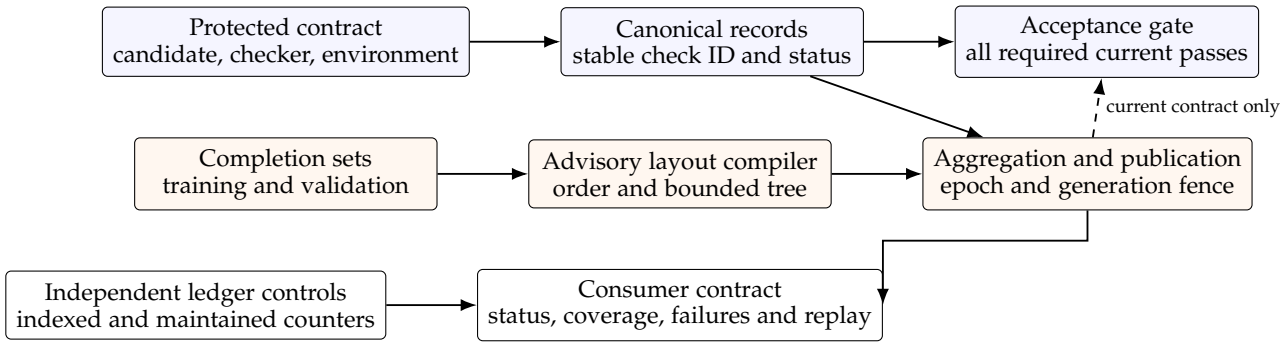


Fig. 1. Semantic authority, adaptive representation and the consumer contract. The compiler cannot modify the protected contract or canonical outcomes. Both ledger controls and the tree answer the same diagnostic requests. The diagram is rendered from the included TikZ source; AI assistance is disclosed in the acknowledgments.

even after a return to identical source bytes. Only a trusted runner can supply final PASS, FAIL or UNKNOWN records. Missing, stale, skipped, incompatible or timed-out evidence does not count as a pass.

A layout $L = (\pi, T)$ maps leaf positions through an exact permutation π of semantic indices. Each internal node's children partition its positional interval without overlap or omissions. Its semantic coverage is $G_L(v)$. Check execution order is independent of layout order. The acceptance predicate is, with $E_e(g_i)$ denoting the current matching trusted status,

$$A(e) = \prod_{g_i \in \mathcal{R}} \mathbf{1}\{E_e(g_i) = \mathsf{PASS}\}. \tag{1}$$

An observed failure permits rejection before the remaining checks execute, but supplies no evidence for the unexecuted remainder.

An internal node with d children emits a packet costing

$$c(d) = H + Sd, \qquad 2 \le d \le b = \lfloor (C - H)/S \rfloor . \tag{2}$$

The implementation uses $H = 512$, $S = 320$, $C = 3072$ bytes and $b = 8$. Mandatory fields exceeding their slots are rejected. The cap is per packet; registry manifests are separate control artifacts whose bytes must be charged.

Let $U_{e,a}$ contain the semantic indices whose final records arrive between two refreshes. The strict-epoch padded cost is

$$\ell_e(L) = W(L) + \sum_a \sum_{v \in \mathrm{int}(T)} c(d_v)\mathbf{1}\{G_L(v) \cap U_{e,a} \neq \varnothing\}, \tag{3}$$

where $W(L) = \sum_v c(d_v)$ constructs an initially unknown tree. We call $U_{e,a}$ a completion hyperedge. This model charges the full initial tree and every touched ancestor, rather than using independent activation probabilities. Persistent refresh models omit a cold build at each event; each actual installation is still cold and priced.

A layout switch from i to j additionally costs $M_{ij} = |\mathrm{manifest}(L_j)| + W(L_j)$ when layouts differ. This is the evaluated uncompressed installation protocol, not a universal network cost. A format using compression or deltas defines a different objective and needs its own measurements. Padded packets, compressed archives, answer traffic, serialized state, RAM and elapsed time remain separate units.

When this objective applies. Neither acceptance nor the evaluated diagnostic API requires a tree. For those fixed-group queries, maintained counters are the supported service choice. Equation (3) instead prices a consumer contract that explicitly requests refreshed hierarchical capsules with the stated slots. A remote incremental auditor subscribing to intermediate subtree counts is a possible contract, not a measured use case or a reason that a flat ledger cannot emit the same stream. Flat deltas, batching and compressed codecs would need equal-service comparisons. HACT's local bindings do not provide Merkle membership proofs or enclave attestation; cryptographic or distributed consumers require additional protocol design and evidence.

3 An Information Limit of Pairwise Summaries

Proposition 1 (Pairwise probabilities omit subtree-hit information)**.** *On three leaves, let P be uniform on $\varnothing, \{0,1\}, \{0,2\}, \{1,2\}$ and Q uniform on $\{0\}, \{1\}, \{2\}, \{0,1,2\}$. Both have singleton probabilities $1/2$, pairwise joint probabilities $1/4$ and Jaccard affinities $1/3$, but the triple is touched with probabilities $3/4$ and 1.*

This parity fact alone does not imply different global optima: another tree might dominate in both worlds. The following theorem rules that out under the affine packet objective.

Theorem 2 (Pairwise information has a sharp optimization penalty)**.** *Let $b \ge 3$ be an integer, $H \ge 0$, $S > 0$, and $\lambda \ge 0$. For $n = b + 1$ labeled leaves and internal arities $2, \ldots, b$, define*

$$J_D(T;\lambda) = \lambda W(T) + \mathbb{E}_{U \sim D} \sum_{v \in \mathrm{int}(T)} c(d_v)\mathbf{1}\{G(v) \cap U \neq \varnothing\}.$$

There are distributions P, Q with identical singleton and pairwise probabilities but disjoint globally optimal tree sets, over every feasible shape and leaf permutation. With $W_2 = 2H + S(b+2)$,

$$J_P^* = (1+\lambda)W_2 - (H + 3S)/4, \tag{4}$$

$$J_Q^* = (1+\lambda)W_2 - (H + 2S)/4. \tag{5}$$

Any deterministic selector observing only those exact pairwise summaries has worst-case regret at least $S/4$. The sharp randomized minimax regret is

$$\frac{S(H+2S)}{4(H+3S)}. \tag{6}$$

Sibling orders are treated as equivalent.

Proof. Append $b-2$ anchor leaves, present in every event, to the two three-leaf parity laws of Proposition 1. Every singleton and pairwise probability agrees, including anchor/parity pairs. An anchor-containing node is always touched; a node containing only parity leaves is touched with probability at least $3/4$.

If I is the number of internal nodes, edge counting gives $W(T) = I(H+S) + Sb$, and the arity cap implies $I \geq 2$. At most two internal nodes contain only the three parity leaves. If there are two, both are binary; if there is one, its arity is at most three. Their total price is at most $2c(2)$. Thus for either law and $I \geq 3$,

$$J_D(T;\lambda) \geq (1+\lambda)W(T) - c(2)/2.$$

A tree with two internal nodes and a parity-pair child costs $(1+\lambda)W_2 - c(2)/4$ under either law. Every $I \geq 3$ lower bound exceeds that pair tree by at least

$$(1+\lambda)(H+S) - c(2)/4 = (3H+2S)/4 + \lambda(H+S) > 0.$$

Such trees are componentwise dominated. With $I = 2$, a child containing an anchor is also dominated by a parity pair. The remaining choices are a parity pair or the full parity triple. A pair is missed with probability $1/4$ under either law. The triple is missed with probability $1/4$ under P and zero under Q. Since $c(3) > c(2)$, P uniquely chooses the triple, whereas Q chooses any of its three pairs. This proves Equations (4)–(5), including construction.

A pairwise-only rule receives identical inputs in both worlds. All other tree classes are dominated, so a randomized rule need only choose the triple with probability q and a pair otherwise. Its two regrets are

$$r_P(q) = S(1-q)/4, \qquad r_Q(q) = (H+2S)q/4.$$

The best deterministic choice is a pair, giving worst-case regret $S/4$. Equating the two affine regrets gives $q^* = S/(H+3S)$ and Equation (6). This mixture attains the lower bound. □

The result covers every affine positive-slot price and every fanout at least three; it is not specific to the artifact's selected byte constants. It concerns an interface exposing exact population pairwise summaries. It does not assert equivalence of finite-sample statistical experiments: joint fluctuations of empirical summary vectors can retain higher-order information. It neither bounds a method that retains full waves nor establishes the prevalence of such laws in software executions. The events describe persistent refresh; a partial event is not asserted to be a complete strict verification epoch.

At these prices the deterministic penalty is 80 bytes; randomized minimax regret is $1440/23 \simeq 62.609$ bytes, attained by choosing the triple with probability $5/23$. Integer byte counts are the implementation case; the theorem also allows real prices. Its zero-header boundary does not change the serializer's stricter positive-header validation.

An independently implemented exact rational reference checks 39 parameter cases, including zero headers, fractional construction and header-dominated prices. It imports no HACT objective or optimizer. The nine-leaf case visits 511 nonempty subset subproblems and 115,462 candidate partitions *across those subproblems*; it finds one P optimum and three Q optima. Twenty-three tests include independently listed four-leaf trees and deliberate formula corruption. These are analytical checks, not additional software workloads.

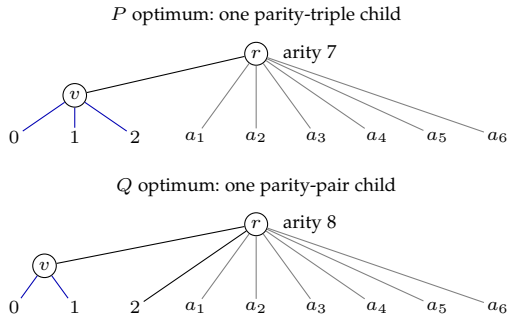


Fig. 2. Nine-leaf witness for Theorem 2, with $H = 512, S = 320, b = 8$. The six anchors a_i are distinct always-present leaves, each a separate root child. Grouping the parity triple costs 3,856/4,224 refresh bytes under P/Q; grouping a pair costs 3,936 under either law. Construction adds 4,224 bytes to both trees.

3.1 The separation also occurs in complete epochs

Persistent refresh is not the only affected regime. Complete epochs partition all obligations into disjoint nonempty waves.

Theorem 3 (Complete-epoch information separation)**.** *For any fanout $b \geq 6$, $S > 0$ and $16/11 < H/S < 28/17$, two laws on six checks partitioned into three nonempty waves have identical population singleton and pairwise completion statistics but disjoint unique global tree optima. Define*

$$\Delta_P = (28S - 17H)/8, \qquad \Delta_Q = (11H - 16S)/5.$$

The sharp deterministic pairwise-only regret is $\min(\Delta_P, \Delta_Q)$; the sharp randomized minimax regret is $\Delta_P\Delta_Q/(\Delta_P + \Delta_Q)$.

Proof sketch; full orbit certificate in the companion analysis. Let $G = \{0,1,2,3\}$, $O = \{4,5\}$. A block's type (g,o) counts its members from G, O. Let $A = (G, \{4\}, \{5\})$, and let B, C, D be uniform over partitions with block-type multisets respectively

$$B : \{(2,1),(2,0),(0,1)\},$$
$$C : \{(2,1),(1,1),(1,0)\},$$
$$D : \{(2,0),(1,1),(1,1)\}.$$

Take $P = 7A/8 + B/8$, $Q = 9A/10 + C/20 + D/20$. Within-G same-wave probabilities are $11/12$, cross probabilities $1/32$, and the O pair probability zero in both laws. Uniformly permuting wave order aligns every window-specific first/second moment.

Contraction strictly improves every nonroot binary or ternary internal node throughout the stated price interval. With only six leaves, this reduces all potentially optimal shapes to a flat root or one internal child. Exact counting over the remaining coverage orbits leaves only flat and the tree grouping G nondominated. Their cost differences are Δ_P and $-\Delta_Q$. Mixing these two actions and equating their regrets gives the stated minimax result. The companion

analysis supplies the contraction inequalities and all orbit coefficients; an independent exact reference checks all 2,752 labeled unordered trees. □

At the implemented prices the two wrong-choice penalties are 32 and 102.4 bytes; randomized minimax regret is $512/21$ bytes. This constructed law has variable-width windows. The next result connects it to fixed-eight batching without claiming a global 24-leaf optimum.

3.2 Fixed-width batches retain a local information gap

Complete-epoch separation also survives the evaluated flush width. The following claim concerns two fixed layouts, rather than a global 24-check optimum.

Proposition 4 (Fixed-eight candidate-class separation)**.** *For $b \geq 6$ and the price interval of Theorem 3, two laws on complete 24-check execution sequences have identical population bivariate completion-rank distributions but opposite preferred choices in a two-layout class under consecutive batches of eight. Their wrong-choice penalties remain Δ_P, Δ_Q, and their class-relative sharp randomized minimax regret is $\Delta_P\Delta_Q/(\Delta_P + \Delta_Q)$.*

Proof sketch; padding identities in the supplement. Add 18 distinct fillers to the three-window six-check construction, uniformly filling each window to eight checks. The probability that an original check shares a window with a named filler, and that two fillers share a window, depends only on the original pairwise same-window probabilities. Thus these probabilities agree under both laws. Randomly permute the three windows and independently shuffle their members. For two distinct checks with same-window probability p, any specified pair of distinct ranks has mass $p/(3 \cdot 8 \cdot 7)$ within a batch and $(1-p)/(3 \cdot 2 \cdot 8^2)$ between batches. All bivariate rank laws agree.

Place the original six-check subtree and an identical filler-only subtree under a common root. Compare a flat original subtree with the subtree grouping G. Root and filler costs cancel within each law, preserving both penalties and the two-action minimax calculation. The supplement gives the full padding lemma, counting argument and exact conformance checks. □

At the implemented prices the penalties are 32 and 102.4 bytes, with randomized regret $512/21$ bytes. The retained reference checks 600 complete 24-check sequences. Neither the rank equivalence nor these finite checks establish global 24-leaf optimality or equality of joint laws of empirical pairwise-summary vectors.

4 Semantics Through Adaptation

A result token carries the candidate epoch, stable ID g_i and a binding to the identified registry, source, checker and environment. It does not use a display position as identity. A layout-specific kernel maps each stable ID to its current position. Migration revokes publication tickets, increments a monotone generation and reconstructs from canonical records. Reusing an ordinal cache entry under another permutation is disallowed.

Identical duplicate evidence is idempotent. Conflicting final records in an epoch revoke authorization: a recorded failure remains a failure, and other conflicts become unknown. A new epoch is required to resolve them. Publication tickets additionally bind the layout hash and generation. Returning to an earlier tree therefore cannot revive its earlier ticket.

Theorem 5 (Layout-independent local publication)**.** *Assume a complete approved registry and dependency identities, a trusted checker executing immutable identified inputs, collision-resistant bindings and atomic local epoch/generation checks. For any valid sequence of layouts, advisory learning outputs and delivered evidence order, authorization implies $A(e) = 1$. Two valid layouts reconstructed from the same canonical record set have identical root pass/fail/unknown counts.*

Proof. A canonical pass requires a current token bound to the same semantic ID and immutable identities. A permutation is a bijection, so reconstruction moves each record exactly once without changing its status. Exact child partitions preserve coverage and counts by induction from leaves to root. Authorization requires root counts $(n, 0, 0)$ together with matching current registry, epoch, layout and generation. Old tickets fail those checks; conflicts prevent a pass. Advisory layout selection cannot change any canonical outcome or approved obligation. □

The guard implements conflict persistence and tickets; the claim is local and conditional. A compromised checker, incomplete suite or omitted dependency invalidates broader claims. There is no attestation or OS sandbox; an external merge service must atomically bind the verified candidate to its integration head.

The implementation hashes protected input *sets* to detect additions, changes and deletions, and checks source manifests around snapshotting and verification. Snapshots and detailed evidence lie outside the candidate's write scope; the worker receives a bounded result card. The artifact tests stale tokens, conflicts, migration, coverage and source mutation. Passing tests do not establish that predicates cover all program behavior.

5 Bounded Full-Wave Compilation

5.1 Exact fitting at a fixed order

For a fixed order π, training epochs define interval weights

$$r_\pi(i,j) = 1 + \frac{1}{m}\sum_{e=1}^{m}\sum_{a}\mathbf{1}\{U_{e,a} \cap G_L(i,j) \neq \varnothing\}. \quad (7)$$

The one charges construction per strict epoch. In a persistent refresh study that term is omitted explicitly. Let $D(i,j)$ be the best subtree cost and $F_k(i,j)$ the best ordered k-component forest. Infeasible states are infinite:

$$\begin{aligned} D(i,i) &= 0, \qquad F_1(i,j) = D(i,j), \\ F_k(i,j) &= \min_{i+k-2 \leq u < j}\{F_{k-1}(i,u) + D(u+1,j)\}, \\ D(i,j) &= \min_{2 \leq k \leq \min(b, j-i+1)}\{r_\pi(i,j)c(k) + F_k(i,j)\}. \end{aligned} \quad (8)$$

The recurrence enumerates every root arity and contiguous partition. Induction on interval length proves exactness for the supplied nonnegative weights, conditional on order. It uses $O(bn^3)$ time and $O(bn^2)$ space. This specializes classical alphabetic-tree optimization [7]; it is not a global permutation optimizer or an optimum for compressed logs.

5.2 A bounded restricted optimizer

The historical practical method fits Jaccard affinities, obtains an average-linkage order and groups consecutive leaves into compact blocks of at most eight. Full-wave validation retains either the incumbent or the learned compact layout. The new optional method refines *maximal existing subtrees* containing at most s leaves. Each selected subtree is disjoint from the others. It keeps the order and that subtree's leaf interval, solves Equation (8) on projected training waves, and accepts a replacement only if the complete integer training objective strictly decreases. All larger ancestors remain unchanged; ties retain the incumbent subtree.

Proposition 6 (Restricted exactness and bounded refinement)**.** *For a fixed order and fixed disjoint refinement intervals, independent exact fitting minimizes training cost over trees obtained by replacing only those subtrees. Retaining the incumbent on ties gives nonincreasing integer training cost. For intervals of sizes $m_i \le s$, the DP work is at most $O(bns^2)$ with $O(bs^2)$ peak DP storage, apart from input projection, cost computation and ordering.*

Proof. All ancestor contributions depend only on the unchanged coverage and arity. The mutable objective is the sum of independent interval objectives, so minimizing each minimizes the restricted sum. The incumbent is feasible in every subproblem. Since $\sum_i m_i \le n$, $\sum_i m_i^3 \le s^2 \sum_i m_i \le ns^2$. Sequential solution gives the stated storage bound. Integer rescoring rejects a numerically nonimproving DP replacement. □

The implementation uses float64 inside the existing recurrence; exactness is to numerical precision. Independent integer rescoring establishes monotonicity, not exact rational optimality for every large instance. Independent literal-tree enumeration checks tiny cases. Every leaf remains present exactly once and all packet arities remain feasible.

The primary frozen cap is $s = 32$; 64 is a secondary sensitivity. A full compact8 ancestor above eight eight-leaf blocks covers $8^2 = 64$ leaves. With $s = 32$, its blocks are solved separately: no replacement can cross a block boundary, merge adjacent blocks or change that ancestor. Cap64 opens this first cross-block topology decision. Ragged boundary subtrees can have other sizes; the actual largest solved interval is reported. This interaction explains why cap32 often ties while the prespecified cap64 sensitivity improves. It does not change the frozen primary endpoint or isolate a higher-order causal effect. Dense pairwise ordering still has quadratic storage and approximately $O(n^2 \log n)$ heap work. It can dominate bounded refinement and is admitted separately under the resource budget.

5.3 A bounded sparse ordering proposal

The post-review prototype stores positive Jaccard pair counts from training waves and proposes a compact order by a deterministic maximum-spanning forest [8] and depth-first traversal. For W waves of width at most w, it uses $O(n + E)$ ordering storage, where $E \le W\binom{w}{2}$ is the number of distinct positive pairs; sorting costs $O(E \log E)$ arithmetic operations. An explicit edge budget rejects oversized inputs. This replaces the dense ordering heuristic, not the full-wave objective or the verifier. Validation chooses between original and sparse compact layouts using exact touched-node costs; the separate ancestor-scoring index takes $O(n \,\mathrm{height})$ references. No cap64 fit is included in the large-registry follow-up.

5.4 An exact global baseline under completion symmetry

The information lower bound is a worst-case statement. A structured null regime admits a global optimum without learning co-activity. Fix nonempty batch sizes $w_1, \dots, w_q$ summing to n. Complete a uniformly random permutation of the registry and partition it into those consecutive batches. For any semantic coverage G of size s, the expected number of touched waves is

$$r_n(s) - 1 = \sum_{a=1}^{q} \left(1 - \frac{\binom{n-s}{w_a}}{\binom{n}{w_a}}\right), \tag{9}$$

where a binomial coefficient with upper argument smaller than its lower argument is zero. The one in $r_n(s)$ charges the cold build. No independence among checks or waves is assumed. The formula also follows from exchangeability conditional on the declared ordered batch sizes.

Let $D(s)$ minimize expected subtree cost for s leaves, using the *full-registry* weights $r_n(s)$. Let $F_d(s)$ minimize the sum of d nonempty child-subtree costs, with infeasible states infinite:

$$\begin{aligned} D(1) &= 0, \qquad F_1(s) = D(s), \\ F_d(s) &= \min_{1 \le t \le s-d+1} \{D(t) + F_{d-1}(s-t)\}, \\ D(s) &= \min_{2 \le d \le \min(b,s)} \{r_n(s)c(d) + F_d(s)\}. \end{aligned} \tag{10}$$

Theorem 7 (Global optimality under exchangeability)**.** *For the declared exchangeable completion law, Equation* (10) *minimizes the expected strict-epoch padded cost over every feasible tree shape and leaf permutation. After computing the weights, it uses $O(bn^2)$ arithmetic operations and $O(bn)$ states. With integer packet prices it can be evaluated exactly using a common denominator and integer numerators; arithmetic bit complexity is additional.*

Proof. Any batch is a uniform w_a-subset of the full registry. Its probability of missing G is the binomial ratio in (9). Linearity of expectation gives each internal node the contribution $r_n(s)c(d)$, depending only on its coverage size and arity. Conditioning on a root's arity and nonempty child sizes leaves the sum of independent optimization subproblems; probabilistic independence is unnecessary. Induction on s proves the forest and tree recurrences enumerate all possible shapes. Relabeling leaves changes no expected contribution, so canonical depth-first labels attain the unrestricted permutation optimum. Every forest transition scans at most s splits for each (d, s), giving $O(bn^2)$ work and $O(bn)$ stored costs/splits. Multiplying all rational weights by their common denominator preserves every comparison and tie. □

Consequently, if a future epoch retains this exchangeable law conditional on the training data and selected layout, no training-dependent selection improves its expected padded cost below $D(n)$. Condition on that selection: each fixed tree costs at least $D(n)$, then average over training. Independent

training and future epochs suffice; unconditional symmetry alone does not exclude training information that predicts the future completion pattern. This gives a concrete null baseline against which to test learning. Graham, Li and Yao already optimize batch-update key trees using cardinality states under independent Bernoulli replacements [6]. Our fixed-size, without-replacement law, affine capped packet prices and per-epoch cold charge specify a different instance; the size-state principle is established. Neither a shuffled dispatch queue nor a good fit to one trace establishes exchangeable completion times. Violating the model removes the global-optimality guarantee, so we also score this baseline on the frozen structured traces and retain its losses. The objective excludes query service, compilation seconds and installation manifests; those remain separate lifecycle ledgers. The model-derived layout is an optional advisory candidate and does not change the acceptance registry or runtime policy.

5.5 Validation and adoption

Training fixes a finite catalogue containing the incumbent first. Validation observes complete waves and selects the minimum padded cost, retaining the incumbent on ties. The historical method remains unchanged; the refined catalogue is optional.

Empirical validation alone gives no deployment guarantee. The following rule uses independent calibration and an explicit shift allowance. Repeated suite runs do not establish those sampling assumptions.

5.6 Conditional adoption with lifecycle costs

A training-fitted catalogue is frozen before independent calibration. All layouts share the registry and packet contract. Relative to incumbent L_0, aggregate packet-price differences by semantic coverage:

$$a_{jG} = \sum_{v:G_{L_0}(v)=G} c(d_v) - \sum_{v:G_{L_j}(v)=G} c(d_v).$$

Shared packets cancel. If $h_e(G)$ counts intersecting waves, the strict-epoch saving $d_j(e) = \ell_e(L_0) - \ell_e(L_j)$ is

$$d_j(e) = a_{j0} + \sum_G a_{jG} h_e(G), \quad a_{j0} = \sum_G a_{jG}. \tag{11}$$

For an epoch partitioning n checks into exactly q nonempty waves of size at most w, with $q \le n \le qw$, a coverage of size s satisfies

$$l_G = \max\{\lceil s/w \rceil, q - (n-s)\} \le h_e(G) \le u_G = \min(s, q). \tag{12}$$

Every untouched wave needs a distinct check outside G; in particular $h_e(\mathcal{R}) = q$. Conservative bounds on the entire declared support are

$$A_j = a_{j0} + \sum_G \min\{a_{jG} l_G, a_{jG} u_G\},$$
$$B_j = a_{j0} + \sum_G \max\{a_{jG} l_G, a_{jG} u_G\}.$$

Thus $d_j(e) \in [A_j, B_j]$ and $R_j = B_j - A_j$. Different coverages' extrema need not be jointly attainable.

Proposition 8 (Exact population score identification)**.** *On finite epoch support Ω, an interface exposing only $\mathbb{E}_P\phi_1, \dots, \mathbb{E}_P\phi_p$ determines $\mathbb{E}_P d_j$ for every law P if and only if $d_j \in \operatorname{span}\{1, \phi_1, \dots, \phi_p\}$ on Ω.*

Proof sketch; full identification argument in the supplement. Span membership gives affine recovery. Otherwise a signed vector orthogonal to $1, \phi_1, \dots, \phi_p$, but not d_j, gives two normalized nonnegative laws with identical exposed moments and different expected scores. □

For a frozen catalogue, exact paired scoring needs only each accumulated integer score and an epoch count: $O(K)$ counters besides coefficients and one epoch's masks. Acceptance records and required raw provenance remain separate. Exact score recovery alone does not imply argmin recovery.

Theorem 9 (Conditional expected lifecycle benefit)**.** *Condition on training and the frozen catalogue of $K \ge 1$ candidates, support, $0 < \delta < 1$, TV radius $0 \le \rho \le 1$, horizon $h \ge 1$ and nonnegative installation costs I_j, all specified before calibration. Suppose the $m \ge 1$ calibration epochs are IID from P and independent of fitting. With paired mean $\bar{d}_j$, define*

$$t_j = R_j \sqrt{\frac{\log(K/\delta)}{2m}},$$
$$g_j = \max\{A_j, \bar{d}_j - t_j - \rho R_j\} - I_j/h. \tag{13}$$

With probability at least $1 - \delta$ over calibration, simultaneously for all candidates and every future law Q on the same support with $\mathrm{TV}(P, Q) \le \rho$, $h\,\mathbb{E}_Q d_j - I_j \ge h g_j$. Adopting only a candidate with $g_j > 0$, otherwise retaining L_0, certifies a conditional positive expected padded-byte benefit.

Proof. One-sided Hoeffding bounds and a union bound give $\mathbb{E}_P d_j \ge \bar{d}_j - t_j$ simultaneously [9], [10]. For $\mathrm{TV}(P, Q) = \sup_E |P(E) - Q(E)|$, a function of range width R_j changes expectation by at most ρR_j. Also $\mathbb{E}_Q d_j \ge A_j$ on the same support. Subtract installation and multiply by h. Zero-width scores are constant, so the conclusion also holds when $R_j = 0$. The supplement gives the full argument and implementation contract. □

This certifies neither realized savings, elapsed time nor acceptance. The sampling design and radius require external justification. Deployment outside the support, refitting on calibration, or repeatedly changing catalogues needs new accounting. The implementation binds the plan to a digest and requires sampling/freeze provenance; missing evidence returns UNKNOWN. Installation is charged once; compilation and consumer utility remain separate admission tests.

Compilation, selection and installation are separately charged. A saving of g bytes per epoch repays installation I only when $hg > I$; including compilation seconds requires a measured unit conversion. Padded bytes imply no token or agent speedup. The supplement's empirical-Bernstein alternative has a finite-range term exceeding the constructed mean saving at $m = 50$; variance sensitivity is not an established remedy in this example.

TABLE 1
Established ingredients and the contribution claimed here.

Established ingredient	Claim under the HACT contract
Alphabetic and batch-update tree DP	Exact fixed-order reference and global size-state optimum under declared exchangeability; both specialize established DP principles.
Pairwise clustering and hypergraphs	Explicit indistinguishable laws with disjoint global optima and sharp regret; fixed-batch local extension.
Epochs, identities and fencing	Conditional composition preserving the approved all-check acceptance predicate during representation migration.
Subtree optimization and validation	Bounded full-wave refinement; conditional paired-cost adoption under externally justified sampling and shift assumptions.
Ledgers, counters and common codecs	Controlled comparison separating representation gains from service costs, including losses.

6 Evaluation Protocol

The evaluation asks four questions:

1) Can the information limit and semantic invariant be checked independently?
2) Does bounded full-wave refinement improve over compact baselines?
3) Does a tree repay its full service costs against strong ledgers?
4) Which observed workload and resource boundaries limit adoption?

6.1 Evidence cohorts and frozen decisions

Freezes are internal source/input records, not external preregistration; cohorts and endpoint roles remain fixed. The compiler study reuses 27 frozen synthetic train/validation/held-out cells and twelve archived candidate traces from three projects, preserving their provenance.

The public cohort is a declared convenience sample: Click 8.2.1, Jinja 3.1.6, Packaging 25.0, Pluggy 1.6.0, attrs 25.3.0 and ExceptionGroup 1.3.0. Tags resolve to commits, tracked files are hash-bound, and imported modules are checked against clean checkouts. Five fresh runs of each unchanged upstream suite are assigned in advance: runs 0–1 training, 2 validation and 3–4 held out. Final phases and arrival order are recorded; at most eight successive completions form a wave, as in the historical protocol. These test executions are not independent source changes, natural-bug repairs or agent episodes.

The registry is a sorted ID set; collection, scheduling and completion orders are retained separately. A collection-order instrumentation correction is recorded before cost analysis, with raw inputs preserved. Skips, xfails and xpasses remain UNKNOWN; ordinary failures are retained, environmental failures disclosed, and exclusions and resource admission explicit.

Methods and receipts are frozen before held-out costs are opened. The six requested projects, five admitted to optimization, are case-level units; checks, cached requests, repeats and replays do not enlarge the sample. We report per-project measurements without population-level superiority tests.

6.2 Controls, estimands and uncertainty

The primary compiler endpoint compares refined and historical compact selection on strict-epoch padded bytes, including construction. Original and pairwise orders are compared before and after refinement at the *same order*, separating topology from ordering without identifying higher-order information as the cause of a gain. The exact parity case checks use of full waves when pairwise affinities agree; it is analytical conformance.

The service endpoint sums local operations over a retained trajectory: construction, eight-completion ingests, query aggregation, encoding and decoding. Input snapshot preparation and the independent oracle are outside both timers; checker execution is separate. Initial, intermediate and final snapshots receive status, failure, unfinished, coverage and replay queries, each cold and repeated warm. This layout-independent sequence is a controlled integration workload, not observed production demand.

Controls are the original indexed ledger and a stronger ledger maintaining fixed-group status counters. Tree and ledger query maintained state; a third computation answers from raw records. Answers, missing-obligation sets and epochs must match or performance promotion halts. Identical bounded pages and deterministic DEFLATE imply equal answer bytes. Serialized state and logical update bytes remain separate from wire traffic and resident memory.

Three whole-trajectory executions use seeded randomized arm order. Timings describe this host and workload, not additional projects. Paired per-case summaries retain unfavorable cases without significance claims from correlated queries. Per-arm timeouts remain UNKNOWN, not zero-cost successes.

6.3 Review-motivated follow-up protocols

Two separately frozen follow-ups address weaknesses found in the original study. Both retain the original layouts and all unfavorable observations. First, each of the five admitted release suites executes under three fixed seeds crossed with two dispatch policies: shuffling test-file groups while preserving within-file order (primary), and shuffling individual items (secondary). These are actual complete pytest processes, not permutations of recorded outcomes. Registry equality, scheduled versus actual completion order, imported source origins and pre/post source hashes are checked. Fixed-eight waves from each completed execution are priced by an independent literal-tree evaluator. No layout is refitted, and a registry, timeout or provenance failure makes a case VOID or UNKNOWN. The method retains both the original compact and historical selected comparators.

Second, byte-count instrumentation in the original service serializes maintained state and records inside the timers. A new eight-arm campaign crosses all four services with original and observer-disabled implementations on the same ten admitted trajectories, with three seeded randomized-order repeats. Observer-disabled subclasses remove serialization used solely to measure byte counters; functional record copies, validation, evidence digests, kernel refresh and packet serialization, caches and answer-codec work remain. The old observations are not retroactively corrected. New semantic/wire oracle checks precede interpretation

of timings. Full duplicate-aware package inventories and source/input hashes are frozen for both follow-ups.

OpenAI Codex supported implementation, proof checking, experiment orchestration and analysis; checks by separate AI agents are automated checks, not human peer review. Numerical tables and plots derive from retained JSON observations through the supplied Python scripts. The v11 figure code was assisted by Codex; the v12 analysis and revision also used Codex. Exact model-version identifiers were not captured in the experiment manifests. The raw observations, code and protocols permit verification without an AI service. No language model participates in the measured service operations.

7 Results and Applicability

7.1 Theory and semantic conformance

The independent rational reference verifies 39 parameter cases for Theorem 2. Independently listed literal trees, partition-count checks and formula-tampering tests reduce shared-code risk. The mathematical proof covers arbitrary admissible fanout and prices; finite enumeration checks its implementation and boundary examples, rather than substituting for the proof. Semantic regression tests cover stale tickets, conflicts, permutations and epoch/generation transitions.

7.2 Historical representation results and new ablation

Frozen clustered synthetic workloads yield held-out padded-byte savings of 23.43%, 10.75% and 5.56% at 128, 512 and 1,024 checks relative to the original balanced baseline. Independent/global controls show essentially no systematic benefit. Re-encoding the twelve archived candidates changes compressed hierarchical streams by 4.36–41.84%; these are within-format results and do not establish service superiority.

The new bounded-refinement experiment is more restrictive than a global optimizer. At cap 32, all 27 historical cells tie the validation-selected compact catalogue. At cap 64, 26 tie and one saves 0.00762% on held-out padded bytes. The isolated sensitivity gain is not a practical breakthrough or a confirmatory higher-order effect. Independent dictionary-tree reconstruction verifies the reported objectives without calling the producer's cost helper. The archived direct-wave heuristic also showed no gain. These results favor retaining a compact incumbent unless independent evidence pays for a more expensive compiler.

7.3 New public-software executions

Thirty fresh process runs on six upstream projects record 151,380 check executions. Table 2 reports one held-out run per project; the second has the same counts. These counts are execution observations, not independent projects. Only five projects (25 runs, 16,640 check executions) are admitted to the compiler comparison. All five have identical completion orders and ordered wave partitions across runs 0–4. Thus the held-out costs measure repetition of observed completion patterns, not prediction of new patterns. attrs' 33 failures concern this environment (including mypy compatibility) and are retained without calling them upstream bugs. Skips, xfails and xpasses remain unknown.

TABLE 2
New complete suites. Counts per held-out run; five runs per project. P/F/U are PASS/FAIL/UNKNOWN.

Project	Checks	P	F	U
Click	761	739	0	22
Jinja	909	909	0	0
Packaging	26,948	26,948	0	0
Pluggy	124	124	0	0
attrs	1,432	1,391	33	8
ExceptionGroup	102	91	0	11

TABLE 3
Held-out padded-byte reduction (%) relative to the historical compact catalogue. Construction is included. UNKNOWN is not a tie.

Project	Cap32 primary	Cap64 secondary
Click	1.493	17.219
Jinja	0.177	15.996
Packaging	UNKNOWN	
Pluggy	0.000	18.695
attrs	0.000	0.000
ExceptionGroup	0.000	18.690

Raw collection order varied in Packaging. Before any layout-cost analysis, a dated correction created separate canonical-registry copies, retaining the raw records and every scheduled/completed ID unchanged. Jinja's missing Trio test dependency was repaired before its five retained runs; the original failed attempt is preserved. Source files remain hash-verified. The original environment identifier binds Python/platform, not all dependencies. A post-hoc inventory records the repair and duplicate package metadata; it cannot retroactively establish a complete contemporaneous environment identity. Each representation comparison still uses the same persisted input trace.

7.4 Held-out topology costs

Table 3 compares frozen refined selections to historical compact selection. The primary cap32 method improves two of five admitted project cases and ties three. The cap64 results were designated secondary before collection; they do not replace the primary endpoint. Both methods are fitted on runs 0–1, selected on run 2 and scored on runs 3–4. All selected layouts were hash-frozen before those held-out costs were opened.

The positive secondary reductions range from 16.00% to 18.69%; attrs retains its incumbent. The gain is a measured topology-refinement effect at the stated order, not a causal demonstration that higher-order information alone explains it. Figure 3 separates this representation endpoint from service time.

Packaging has 26,948 declared checks. One dense float64 affinity matrix alone would require about 5.81 GB (5.41 GiB); dictionary/heap clustering needs more. The compiler therefore records optimization UNKNOWN under its 2 GiB campaign budget, retains every ID and supplies an explicitly labeled compact fallback only for conformance/scaling records. It is excluded from the five-project compiler comparison rather than being counted as a tie.

TABLE 4
Local service operation time (ms), mean of two per-case medians over three whole-trajectory timing repeats. Builds, all ingests, queries and codec work are included. Layout fitting is reported separately. No network or agent inference occurs.

Project	Indexed	Counters	HACT32
Click	392.1	422.4	996.3
Jinja	555.0	527.4	1236.8
Pluggy	38.5	37.6	56.4
attrs	1067.1	1169.5	2833.9
ExceptionGroup	30.2	29.1	41.8

TABLE 5
Primary cap32 padded-byte reduction (%) under controlled dispatch shifts relative to historical compact selection. Ranges span the three fixed seeds; negative values are losses. No refitting.

Project	File groups	Individual items
Click	-4.85 to -3.33	-9.01 to -8.94
Jinja	0.15 to 0.17	-0.12 to -0.07
Pluggy	0.00	0.00
attrs	0.00	0.00
ExceptionGroup	0.00	0.00

7.5 Transfer to historical-source regressions

An additional frozen plan selects eleven real upstream fix commits by syntax and history, transplants their parent production modules into isolated release snapshots, and runs unchanged release tests with stop-on-first-failure. Nine cases execute; eight preserve the release registry. Two startup incompatibilities and one changed semantic ID set are VOID for paired analysis. One matching Packaging case remains UNKNOWN because fitting was not admitted. The seven measured transfers yield two primary wins, three ties and two losses. In particular, Click's primary layout costs 2.09% and 2.58% more than the historical compact layout on its two partial trajectories. The retained all-pass Jinja case is unrevealing, rather than excluded.

These are exploratory source transplants under a newer fixed test contract, not executions of each native historical parent suite or a certified natural-bug benchmark. They share project tests and supply no extra independent project replication. No source-transfer outcome changes the frozen layout. The losses demonstrate that training monotonicity and a successful repeated-snapshot holdout do not ensure benefit after a source or stopping-pattern change.

7.6 Whole-trajectory service accounting

The original comparison admits ten traces from five projects at the fixed 2,048-check service budget; both complete Packaging inputs remain UNKNOWN without subsampling. It executes four arms, three randomized-order repeats and cold/warm queries at initial, intermediate and final snapshots. All 5,040 service/oracle answer checks agree, including exact epoch and missing-check disclosure. Each arm delivers the same complete answer bytes under the common deterministic codec.

These original timings include observational serialization for byte counters, including whole-state serialization at every ingest. They measure the instrumented prototype rather than isolated functional service costs. All 30 matched timing-repeat HACT32/counter ratios exceed one (range 1.216–2.850); repeated timings are descriptive, not independent project replications. The separately frozen observer ablation below measures the effect of this instrumentation.

The selected HACT service is slower than the maintained-counter ledger in every admitted project case. Its ratio of mean case-median operation times ranges from 1.44 to 2.42 on this host. Additional layout fitting and manifest admission cannot turn those observed service losses into a positive end-to-end speedup. The table times precompiled service execution and does not hide fitting inside common checker time. Compiler receipts retain separate one-time fit/selection measurements; their concurrent collection load makes them descriptive rather than controlled speedup estimates. The outcome supports a maintained ledger as the initial service baseline for these requests, while preserving the distinct within-format packet savings above.

7.7 Controlled dispatch-shift transfer

The new campaign contains 36 complete suite executions and 20,580 check executions. Six initial ExceptionGroup runs are VOID for cost analysis: the profiler observed an ignored build-generated version module absent from the frozen tracked-source inventory. Their raw records are retained. A separate child protocol freezes that module before rerunning all six predeclared schedules, without changing tests, seeds, layouts or endpoints. The resulting admitted comparison has 30 executions and 19,968 checks over five projects; it does not silently reclassify the original exclusions. All admitted runs retain the original registry and status counts, including attrs' environment-related failures.

Against the historical selected comparator, file-group shuffling gives three primary wins, nine ties and three losses; item shuffling gives zero wins, nine ties and six losses. The comparison unit here is a scheduled process, not an independent project. The small Jinja file-group gain survives, whereas Click loses under both shifts. The three ties are projects whose primary selection did not change. This directly limits the inference from the repeated-pattern holdout. Every seed, all four frozen layouts, the original compact comparison and secondary cap64 prices are retained in the artifact. Neither the intervention nor the analytical witness identifies natural deployment prevalence or a causal higher-order benefit.

7.8 Service costs with byte observers disabled

The separate ablation completes 240 arm executions: ten held-out traces, three repeats and eight arms. All 10,080 raw-oracle/common-codec answer checks agree; the two complete Packaging traces remain UNKNOWN under the unchanged service budget. Table 6 compares the same primary layout to the maintained-counter ledger within each new timing block.

Removing observational serialization reduces absolute service time but does not reverse the comparison. Primary HACT is slower than counters in every paired timing block under both instrumentation states. With observers disabled, the project ratios of mean case medians range from 1.61 to

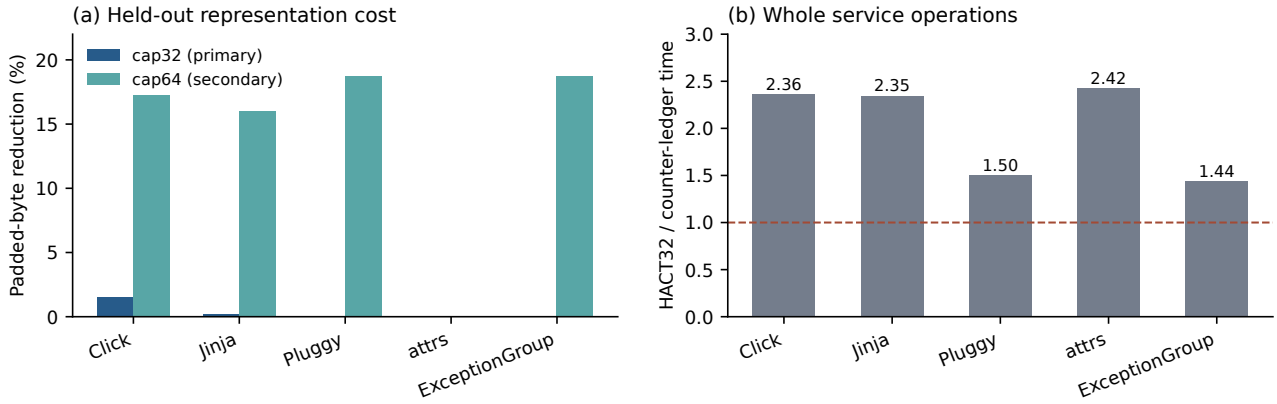


Fig. 3. Different endpoints lead to different adoption decisions. Left: primary and prespecified secondary held-out padded-byte savings. Right: original instrumented precompiled service time of primary HACT divided by the maintained-counter ledger; values above one are slower. Packaging is UNKNOWN under resource admission and is not plotted as zero.

TABLE 6
Observer ablation. Disabled columns are mean case-median operation times (ms); ratios compare HACT32 with counters in the same campaign. Off=byte observers disabled (HACT32 ms / Counter ms at left); on=original instrumentation in separate paired executions.

	Observers off (ms)		HACT/counter	
Project	Counter	HACT32	Off	On
Click	137.9	429.2	3.11	2.16
Jinja	140.3	446.1	3.18	2.23
Pluggy	23.0	37.1	1.61	1.45
attrs	261.3	924.9	3.54	2.64
ExceptionGroup	20.3	39.0	1.92	1.43

3.54; individual paired-block ratios range from 1.135 to 4.630. These are descriptive host-specific ranges, not confidence intervals. The disabled variant retains functional packet serialization, evidence digests, record copies, validation, caches and codec round trips. It is a controlled prototype ablation, not an optimized production implementation. The negative service result therefore survives the identified observer-cost confound for this demand.

7.9 Exact scoring interfaces and conditional adoption

Proposition 8 separates two requirements: retaining enough information to fit a new layout, and scoring a frozen catalogue. On all 90 three-block partitions of six checks, an exact rational calculation gives rank 16 for the constant and fifteen pairwise same-window features, rising to 17 after appending the grouped-versus-flat paired score. A single additional score feature suffices for this fixed comparison. The rank calculation verifies the scoring statement; the earlier global-optimum theorems supply the stronger optimization obstruction.

The independent literal-tree scorer agrees with the paired implementation and structural envelope in 247,680 comparisons: all 2,752 labeled unordered six-leaf trees against flat, across all 90 complete three-window partitions. The checks include arbitrary display orders; four-leaf regressions also enumerate every feasible width/window-count combination. These are exact analytical conformance checks, not new software projects.

TABLE 7
Synthetic conditional adoption counts. Repeated prefixes are dependent within seed. Under Q, neither bound adopts at any listed size.

Calibration m	P: paired	P: whole	Q: paired
100	0/20	0/20	0/20
1,000	0/20	0/20	0/20
10,000	1/20	0/20	0/20
100,000	20/20	0/20	0/20

For the complete-epoch witness at the implemented prices, the grouped layout saves an expected 32 bytes under P and loses 102.4 under Q. Its paired support interval is $[-3328, 256]$, of width 3,584 bytes; bounding both whole costs separately gives width 22,784. Their squared width ratio is approximately 40.41, a bound-dependent sample-complexity comparison, not measured task acceleration.

We fix the two layouts, $\delta = 0.05$, zero shift, a 1,000-epoch horizon and 3,684 additional pre-horizon installation bytes before a seeded synthetic calibration study. Twenty seeds per law use nested prefixes of 100,000 draws. Table 7 reports conditional-adoption counts for Equation (13); whole-cost bounds use the same observed paired means and installation ledger. The direct streaming API agrees with the intended positive scenario and retains the incumbent when the declared TV radius rises to 0.01. No population significance or empirical confidence coverage is inferred from these twenty seeds.

Even with infinite calibration and no installation charge, a mean saving of 32 bytes cannot make this lower bound positive for $\rho \geq 32/3584 = 1/112$. This is the limit of this conservative certificate, not a sharp minimax drift tolerance or an estimate of software-workload drift. All five admitted public-project rows remain UNKNOWN: repeated snapshots and controlled schedule interventions supply neither the required independent natural calibration nor an externally justified deployment TV bound. Packaging's resource exclusion is retained separately. This new analysis neither changes the historical 3/18/9 shift outcomes nor establishes a service advantage over maintained counters.

TABLE 8
Exact symmetric-model baseline versus frozen full-wave cap32. Savings (%) retain both successful and unfavorable dispatch policies.

Project	Checks	Item shuffle	File groups
click	761	+15.67	-7.29
jinja	909	+7.22	-11.50
pluggy	124	+5.28	-15.77
attrs	1,432	+8.26	-23.77
exceptiongroup	102	+7.00	-10.60

7.10 A stronger symmetric-model baseline

The exact size-state solver matches independent all-tree enumeration in 70 combinations of size, fanout and flush width, totaling 74,245 feasible tree comparisons for $n \leq 6$. Its reference averages actual completion permutations rather than importing the hypergeometric formula. Additional regressions cover every small positive batch-size composition, arbitrary leaf orders, three packet-price regimes and exact reconstruction. This checks the model-specific global optimum, not the symmetry of software data.

We then fit this baseline to registry size and the declared eight-record flush only, using canonical semantic order and no training traces. This retrospective comparison re-encodes the same 25 original executions and 30 admitted dispatch interventions; it adds no checker execution. The baseline beats the frozen cap32 full-wave layout on all 15 item-shuffled traces and loses on all 15 file-group traces. Table 8 uses per-project ratios of summed bytes across three retained seeds per policy. Positive values mean fewer padded bytes than the frozen full-wave layout. Policy/seed repetitions are not independent projects.

On the original repeated-order holdouts, the model baseline instead costs 21.26–62.13% more than frozen full-wave selection. Packaging remains outside the 2,048-check fitting budget. These outcomes identify complementary regimes: size-only topology is a strong control for dispersed completions, while learned locality can matter for structured groups. Shuffled dispatch does not prove exchangeable completion times. The baseline's smaller packet cost in one controlled regime does not overturn the counter-ledger service result or establish a net lifecycle benefit on natural consumer demand.

7.11 Post-review resource and diagnostic follow-ups

The frozen cap64 layouts, rescored on the same 30 admitted dispatch traces, win 12 file-group cases and tie three against the historical compact comparator; under item shuffling they lose 12 and tie three. Against the symmetric baseline, cap64 wins all 15 file-group cases and loses all 15 item-shuffled cases. Thus its larger local search budget improves the grouped-completion regime without removing the transfer boundary. The supplement reports every project's three-seed ranges.

A new sparse compact compiler admits the complete Packaging registry: 26,948 checks, 94,350 positive pairs, 319.6 MiB peak process RSS under a 2 GiB address-space limit. Fit plus validation takes 1.37 seconds after input loading. Training uses runs 0–1 and validation run 2; the selected compact layout costs 70,116,096 rather than 111,199,936 padded bytes on each retained holdout (36.95% less). This is a post-review replay on repeated completion patterns, not a new suite execution, cap64 fit or large-registry service trial. Historical Packaging exclusions remain unchanged.

Exclusive process-CPU profiles on the ten admitted cases locate tree functions at 9.0–58.4%, JSON work at 15.4–35.2%, and hashing at 0.9–1.9% of HACT's profiled operation CPU. All 840 paired-arm answer checks agree. These cProfile diagnostics exclude input/oracle work and retain the original unprofiled Table 6 estimand. Allocation is not separately isolated; the supplement gives disjoint categories, counter profiles and the dominant functions.

8 Related Work

Test prioritization, regression selection and self-adjusting computation optimize executed and recomputed work [11], [12], [13], [14]. HACT leaves the registry and schedule to the runner and requires the entire declared contract; it claims no superiority to regression selection.

Graham, Li and Yao price touched ancestors in batch-update key trees and derive an $O(n)$ solver for independent Bernoulli replacement [6]. Our exact baseline in Section 5.4 instead uses fixed-size without-replacement batches, affine capped packets and cold-build charges; the information separation concerns correlated laws outside that regime. Alphabetic trees supply the fixed-order dynamic program [7], and pairwise seriation supplies an ordering proposal [15]. Hierarchical clustering can price pairwise similarities [16] or three-hypergraph objectives [17]; hypergraph partitioning also prices collective communication [18]. HACT differs in arity prices, touched-node objective, unrestricted shapes and lifecycle costs; hardness and approximation guarantees do not automatically transfer.

Authenticated adaptive structures optimize access and proof costs [19]; HACT's local bindings are not proofs, signatures or attestations. Verification-first engineering and trace assurance govern actions and checks [4], [5]; HACT isolates their accumulated representation. Optional migration policies specialize switching-cost expert methods; the historical manuscript retains their assumptions and negative comparisons. Batched telemetry and compression remain necessary controls. Our adoption rule specializes independent calibration and simultaneous risk testing [10] to packet-cost differences, semantic coverage cancellation, declared completion support and separately priced installation. Its Hoeffding and total-variation arguments are standard.

9 Threats to Validity and Deployment Guidance

The lower bound is a worst case; the exchangeable optimum requires its completion law. Bounded refinement depends on its interval class and numerical precision. Byte constants, batching, proposal budgets and codecs can change the preferred layout. Neither the population argument nor the empirical item/file reversal estimates its regime's prevalence.

The six-project convenience sample includes Packaging, which supplies 89.0% of initial executions but is excluded from dense fitting and service replay. The compiler cohort

has 25 runs and 16,640 checks with repeated completion patterns. Dispatch interventions and bug transplants are controlled changes, not independent projects or natural agent workflows; environmental failures remain disclosed. The sparse follow-up admits Packaging within the memory budget, but retains repeated patterns and excludes large-scale service; scaling to 10^5 tests remains untested.

Maintained counters are supported for the evaluated fixed-group requests; hierarchical consumers must justify equal-service lifecycle savings. Profiled CPU shares are not causal shares of elapsed overhead. Natural demand, live-agent productivity, billing and network benefits remain open. The cohorts lack independent IID calibration and a justified deployment radius for the adoption certificate. Safety requires trusted checking and atomic local fencing; test adequacy and an atomic external integration commit remain separate obligations.

10 Conclusion

Joint completions contain information exact pairwise summaries can lose; under exchangeability, cardinality suffices for a global optimum without learning. Adaptive HACT keeps representations outside semantic authority. Its symmetric baseline and bounded refiner show complementary strengths on dispersed and grouped completions, with unfavorable cases and ledger controls retained. The adoption rule accounts for sampling uncertainty, declared shift and installation bytes. Retain full completion sets for the stated objective and validate expected deployment changes: dispatch losses reject universal transfer, while the observer-disabled comparison supports maintained counters for the evaluated requests. Retain the incumbent unless independent lifecycle evidence justifies a change.

Data and Code Availability

The artifact retains v11–v14 evidence and separately frozen v15 follow-ups, versioned observations, references and audits. Historical releases are at https://github.com/xuda1979/future-code/tree/main/hact-paper/Adaptive_HACT_v5. The historical release manifest binds its source and PDF bytes; it is not a signature. The artifact includes historical analysis, raw execution receipts, a reproduction guide and a claim-to-evidence map. A hash-bound offline archive restores observation paths for verification and audits, preserving exclusions and outcomes.

Acknowledgments

OpenAI Codex (October 2026) assisted with prose throughout the article, proofs and examples in Sections III–V, TikZ source for Figures 1–2, and experiment, reference and audit code for Sections VI–VII, including drafting, derivation, implementation, execution and automated checks. Separately implemented checks reduce shared-code risk but are AI-assisted, not independent human peer review. The authors remain responsible for the content, correctness and citations.

References

[1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *International Conference on Learning Representations*, 2024. https://arxiv.org/abs/2310.06770.

[2] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv:2405.15793, 2024. https://arxiv.org/abs/2405.15793.

[3] X. Wang et al. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. arXiv:2407.16741, 2024. https://arxiv.org/abs/2407.16741.

[4] Q. Lu, A. E. Hassan, D. Lo, and T. Zimmermann. Engineering Agentic Systems: Toward Verification-First AIware Engineering. *IEEE Software*, 43(5):22–29, 2026. https://doi.org/10.1109/MS.2026.3697911.

[5] C. Paduraru, P.-L. Bouruc, and A. Stefanescu. A Trace-Based Assurance Framework for Agentic AI Orchestration: Contracts, Testing, and Governance. arXiv:2603.18096, March 2026. https://arxiv.org/abs/2603.18096.

[6] R. L. Graham, M. Li, and F. F. Yao. Optimal Tree Structures for Group Key Management with Batch Updates. *SIAM Journal on Discrete Mathematics*, 21(2):532–547, 2007. https://doi.org/10.1137/06064929X.

[7] H. Vaishnav and M. Pedram. Alphabetic Trees: Enumeration and Optimization with Applications to VLSI Design. Undated author-hosted manuscript, Sections 4.1–4.4. https://www.mpedram.com/Papers/alp-trees-journal.pdf.

[8] J. B. Kruskal, Jr. On the Shortest Spanning Subtree of a Graph and the Traveling Salesman Problem. *Proceedings of the American Mathematical Society*, 7(1):48–50, 1956. https://doi.org/10.1090/S0002-9939-1956-0078686-7.

[9] W. Hoeffding. Probability Inequalities for Sums of Bounded Random Variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963. https://doi.org/10.1080/01621459.1963.10500830.

[10] A. N. Angelopoulos, S. Bates, E. J. Candès, M. I. Jordan, and L. Lei. Learn then Test: Calibrating Predictive Algorithms to Achieve Risk Control. arXiv:2110.01052v5, 2022. https://arxiv.org/abs/2110.01052v5.

[11] G. Rothermel, R. H. Untch, C. Chu, and M. J. Harrold. Prioritizing test cases for regression testing. *IEEE Transactions on Software Engineering*, 27(10):929–948, 2001. https://doi.org/10.1109/32.962562.

[12] M. Gligoric, L. Eloussi, and D. Marinov. Practical regression test selection with dynamic file dependencies. *Proceedings of ISSTA*, 2015. https://users.ece.utexas.edu/~gligoric/papers/GligoricETAL15Ekstazi.pdf.

[13] U. Acar, G. Blelloch, M. Blume, R. Harper, and K. Tangwongsan. A Library for Self-Adjusting Computation. *Electronic Notes in Theoretical Computer Science*, 148:127–154, 2006. https://doi.org/10.1016/j.entcs.2005.11.043.

[14] A. Mokhov, N. Mitchell, and S. Peyton Jones. Build Systems à la Carte. *Proceedings of the ACM on Programming Languages*, 2(ICFP), Article 79, 2018. https://doi.org/10.1145/3236774.

[15] J. E. Atkins, E. G. Boman, and B. Hendrickson. A Spectral Algorithm for Seriation and the Consecutive Ones Problem. *SIAM Journal on Computing*, 28(1):297–310, 1998. https://doi.org/10.1137/S0097539795285771.

[16] S. Dasgupta. A cost function for similarity-based hierarchical clustering. *Proceedings of STOC*, pages 118–127, 2016. https://doi.org/10.1145/2897518.2897527.

[17] V. Chatziafratis, R. Niazadeh, and M. Charikar. Hierarchical Clustering with Structural Constraints. *Proceedings of ICML*, PMLR 80:774–783, 2018. https://proceedings.mlr.press/v80/chatziafratis18a.html.

[18] P. A. Papp, G. Anegg, and A. N. Yzelman. Partitioning Hypergraphs is Hard: Models, Inapproximability, and Applications. *Proceedings of SPAA*, 2023. https://doi.org/10.1145/3558481.3591087.

[19] Z. Shangguan, A. Yaish, and D. Malkhi. Authenticated Data Structures for Dynamic Workloads. arXiv:2608.25206v1, August 2026. https://arxiv.org/abs/2608.25206v1.